CLAUDE CODE 구현 가이드

콜센터 실시간 관제 시스템

가상 고객센터를 대상으로 Claude Code와 함께 만드는 관제 대시보드

기본 개념 · 시스템 설계 · STEP 0~11 단계별 프롬프트

가상 콜 시뮬레이터, KPI 엔진, 신호등 대시보드, 3분 지속 경보, Erlang C 인력 산정, AI 관제 요약까지
수강생은 프롬프트를 붙여 넣고 결과를 확인한다. 설치·실행·수정은 Claude Code가 한다.

핵심

관제 시스템은 지표 계산, 경보, 조치 연결의 세 층으로 만든다

이 덱이 끝나면 노트북 한 대에서 돌아가는 관제 대시보드가 남는다

1

지표는 매뉴얼 정의대로 계산하고 손계산으로 검증한다

서비스레벨의 분모에 포기 콜을 넣는 것처럼, 정의 한 줄이 숫자를 바꾼다.

2

경보는 임계치와 지속 조건을 함께 건다

녹·황·적 신호등과 3분 지속 조건이 있어야 관제사가 예외만 보고 움직인다.

3

조치는 매뉴얼 절차 번호에 묶는다

Erlang C가 부족 인원을 말하고, AI 요약은 매뉴얼 절차 번호를 근거로 단다.

수강생은 붙여 넣고 확인한다 — 내려받기·설치·실행·오류 수정은 Claude Code에게 시킨다

01

기본 지식

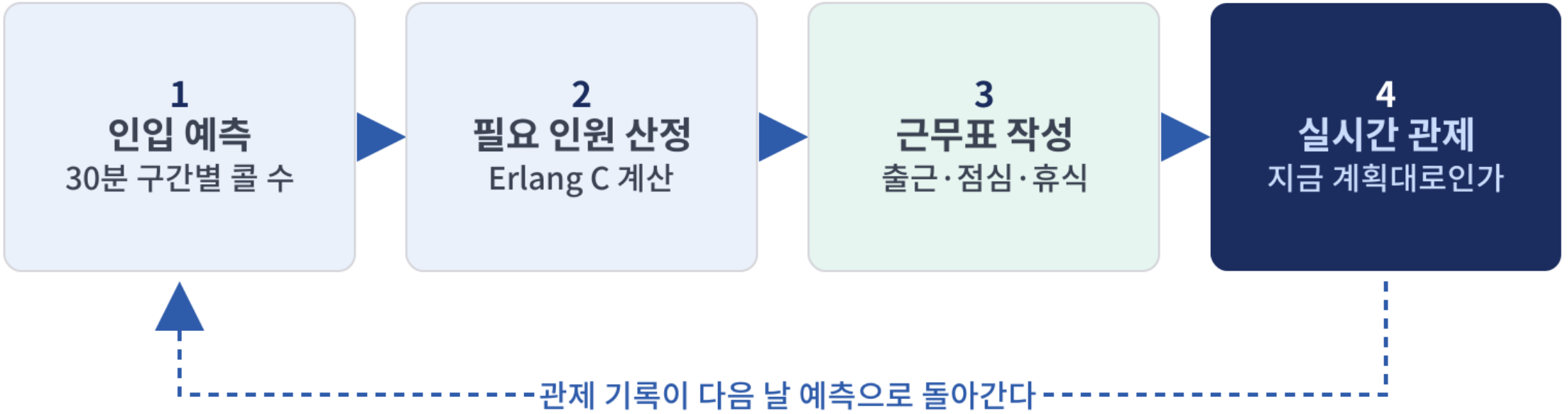
콜센터 관제의 기본 개념

관제가 무엇을 지키는 일인지, 어떤 지표로 무엇을 재는지 먼저 정리한다

실시간 관제는 오늘의 계획이 맞는지 분 단위로 지키는 일이다

WFM(Workforce Management) = 예측 · 인원 산정 · 근무표 · 관제를 아우르는 인력 운영 체계

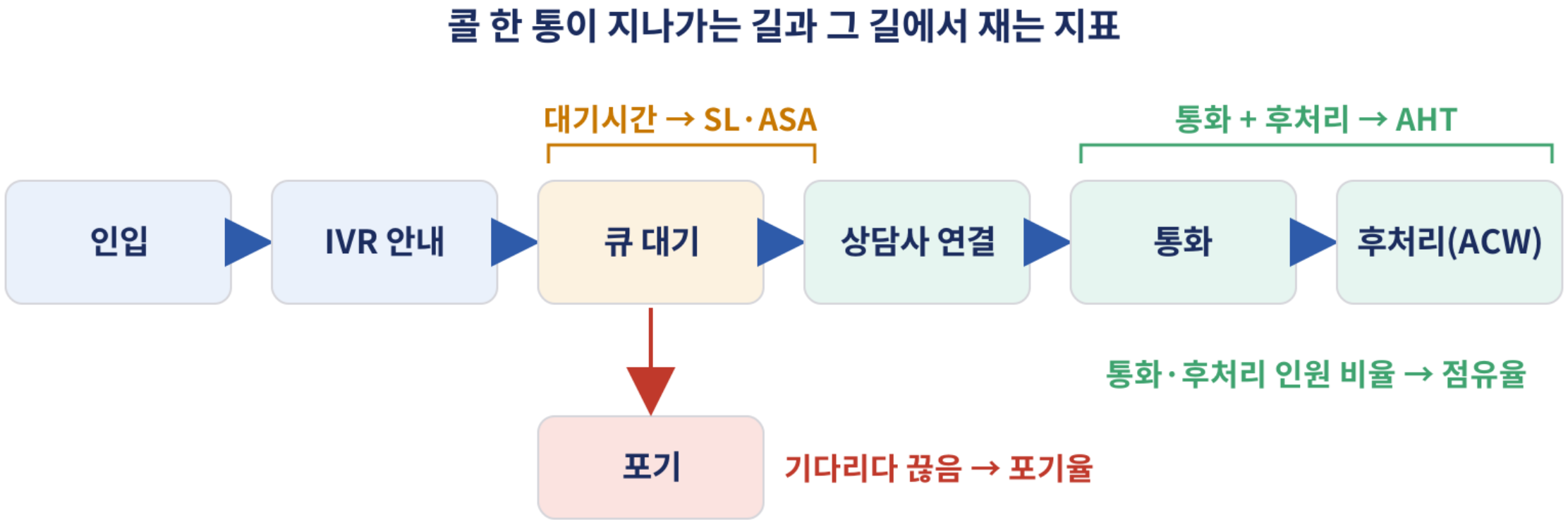
인력 운영(WFM)은 네 단계로 돌고, 관제는 마지막 단계다



1~3단계는 '전날까지의 계획', 4단계는 '오늘 이 순간의 대응'이다

콜 한 통의 경로마다 재는 지표가 다르다

IVR = 번호를 누르게 하는 자동응답, ACW(After Call Work) = 통화 뒤 상담 내용을 입력하는 후처리



KPI ①

서비스레벨의 분모에는 포기 콜을 넣는다

SL(Service Level) — 80/20은 '콜의 80%를 20초 안에 받는다'는 목표다

$$\text{서비스레벨} = \frac{\text{기준시간 이내 응대 콜}}{\text{응대 콜} + \text{포기 콜}}$$

$$\text{포기율} = \frac{\text{포기 콜}}{\text{응대 콜} + \text{포기 콜}}$$

포기 콜을 빼면 생기는 착시

- 응대 80콜 중 64콜이 20초 안, 포기 20콜
- 포기를 빼면 64/80 = 80%로 녹색
- 포기를 넣으면 64/100 = 64%로 적색

그래서 매뉴얼은

- 고객이 오래 기다리다 끊을수록 SL이 좋아 보이는 구조를 막는다.
- 센터 SL은 콜마다 자기 큐의 기준 초를 적용해 합산한다.

점유율은 상담사가 숨 돌릴 틈이 있는지를 잰다

ASA = 평균 응답속도, AHT = 평균 처리시간(Average Handle Time)

$$\text{ASA} = \frac{\sum \text{응대 콜의 대기시간}}{\text{응대 콜 수}} \qquad \text{AHT} = \overline{\text{통화시간} + \text{후처리시간}}$$

$$\text{점유율} = \frac{\text{통화 인원} + \text{후처리 인원}}{\text{대기 인원} + \text{통화 인원} + \text{후처리 인원}}$$

읽는 법

- 휴식·오프라인·장애 이탈 인원은 분모에서 뺀다. 응대할 수 있는 사람 중 몇 %가 일하는 중인지를 본다.
- 점유율이 92%를 넘는 상태가 길어지면 연속 응대로 실수와 이직이 늘어난다. 그래서 상한을 둔다.

관제 지표는 후행 지표와 선행 지표로 나뉜다

후행 지표는 최근 30분 결과의 평균, 선행 지표는 지금 이 순간의 값이다

지표	무엇을 묻나	녹색 기준	성격
서비스레벨	기준시간 안에 연결됐나	80% 이상	후행
포기율	기다리다 끊은 고객은 얼마인가	5% 이하	후행
평균 응답속도	평균 몇 초 기다렸나	15초 이하	후행
대기 콜	지금 몇 명이 기다리나	5건 이하	선행
최장 대기	가장 오래 기다린 고객은 몇 초째인가	60초 이하	선행
점유율	상담사에게 쉴 틈이 있나	88% 이하	선행

후행 지표만 보면 늦다. 선행 지표가 먼저 움직이는 것은 STEP 8에서 직접 확인한다

Erlang C는 부하와 인원으로 서비스레벨을 계산한다

대기가 생기는 콜센터에서 필요 인원을 구하는 표준 대기행렬 공식이다

$$\underbrace{A}_{\text{부하(얼랑)}} = \underbrace{\lambda}_{\text{초당 인입}} \times \underbrace{\mathrm{AHT}}_{\text{평균 처리시간}} \qquad 160\text{콜} \times 280\text{초} \,/\, 1800\text{초} \approx 24.9$$

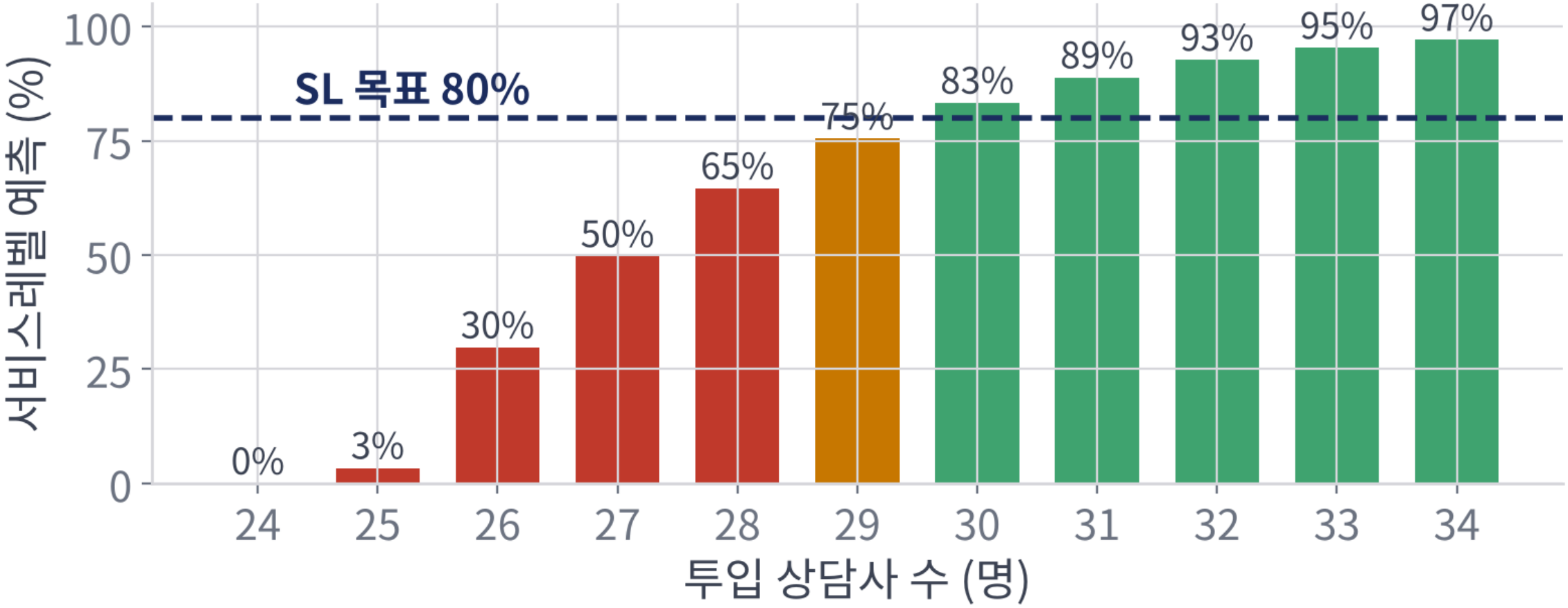
$$P_{\text{대기}} = \frac{\dfrac{A^N}{N!}\dfrac{N}{N-A}}{\displaystyle\sum_{k=0}^{N-1}\frac{A^k}{k!} + \frac{A^N}{N!}\frac{N}{N-A}} \qquad \text{서비스레벨} = 1 - P_{\text{대기}}\, e^{-(N-A)\,T/\mathrm{AHT}}$$

N : 상담사 수 A : 부하(얼랑) T : 기준 초(20초) $P_{\text{대기}}$: 기다리게 될 확률

부하 24.9얼랑 = 쉬지 않고 받아도 24.9명이 필요하다는 뜻이다. 대기를 줄이려면 그보다 더 있어야 한다

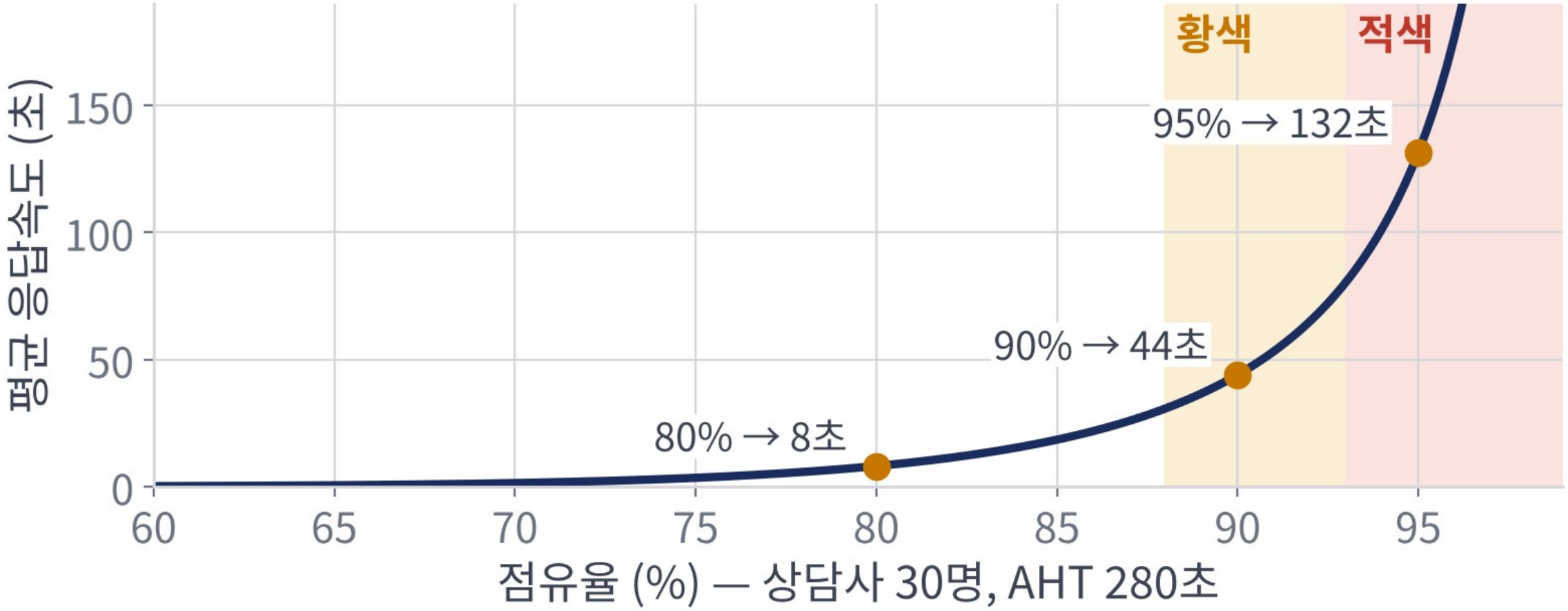
상담사 몇 명 차이가 서비스레벨을 수십 %p 바꾼다

30분 160콜 · AHT 280초 · 기준 20초 — 25명이면 3%, 30명이면 83% (Erlang C 계산)



점유율이 90%를 넘으면 대기시간이 급격히 늘어난다

상담사 30명 고정, 인입만 늘린 경우 — 80%에서 8초, 90%에서 44초, 95%에서 132초



관제가 막아야 하는 장면은 세 유형으로 반복된다

이 실습의 시나리오 버튼은 세 장면을 가상으로 재현한다

예측된 봉우리

결제일·월요일 오전

근무표가 대비했어도 점심 순번이 봉우리와 겹치면 SL이 꺾인다.

→ 실습: 정상 시나리오의 12시 구간

예측 못 한 급증

사고·장애 공지·문자 오발송

2014년 카드사 고객정보 유출 직후 재발급 문의가 몰려 연결이 어렵다는 보도가 이어졌다.

→ 실습: 인입 급증(1.8배)

인력이 사라짐

로그인 장애·전화기 고장

인입은 평소대로인데 받을 사람이 줄어든다. SL은 한참 뒤에야 나빠진다.

→ 실습: 상담사 8명 이탈

첫째는 근무표가, 둘째와 셋째는 실시간 관제가 막아야 하는 장면이다

관제 화면은 무엇이 나쁜지에서 왜 나쁜지로 내려가며 읽힌다

관제사는 여러 화면을 동시에 본다. 5초 안에 상태가 읽혀야 한다

① 신호등

맨 위 KPI 타일

숫자보다 색이 먼저 읽힌다. 녹·황·적 세 가지만 쓴다.

매뉴얼 3장 임계치

② 원인

큐 표 · 상태판

어느 큐가 밀리는지, 누가 이탈했는지를 아래에서 찾는다.

큐별 SL, 상담사 30장

③ 예외만

지속 경보

3분 이어진 것만 경보로 올려 깜박임을 없앤다.

매뉴얼 3장 지속 조건

④ 다음 행동

인력 산정 · AI 요약

부족 인원과 매뉴얼 절차 번호로 할 일을 정한다.

매뉴얼 4장 절차

실습 대시보드도 이 순서대로 위에서 아래로 쌓는다

02

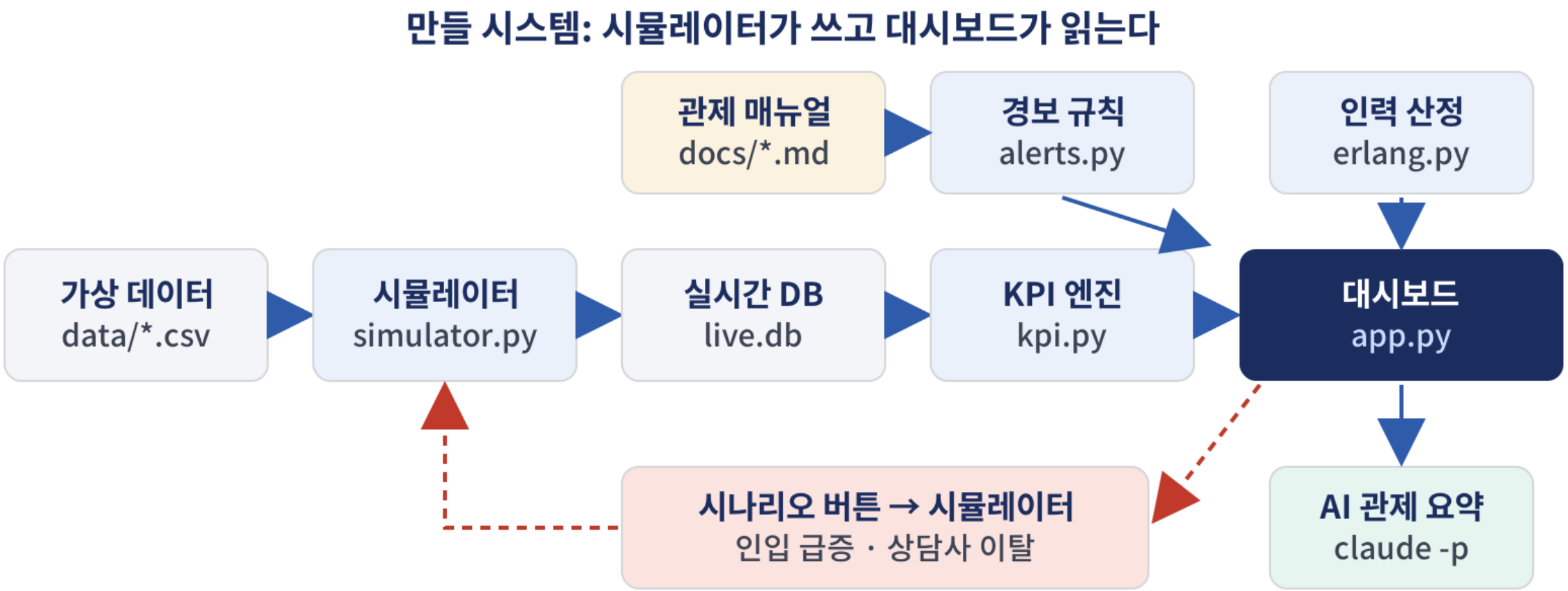
설계와 실습 준비

시스템 설계와 실습 키트

무엇을 만들지, 어떤 가상 데이터와 매뉴얼로 만들지 정한다

시뮬레이터가 DB에 쓰고 대시보드가 DB를 읽는다

실제 전화 시스템(CTI) 대신 가상 시뮬레이터를 쓴다. 외부 서버와 API 키가 필요 없다



실습 키트는 가상 데이터 다섯 개와 문서 두 개로 이루어진다

CallCenter_Monitoring_Kit.zip — 가상의 '하늘카드 고객센터' 기준

파일	내용	어디에 쓰나
data/queues.csv	큐 3개의 우선순위·SL 목표·AHT·인내시간	시뮬레이터, KPI
data/agents.csv	상담사 30명(가명)의 팀·스킬·근무·점심	시뮬레이터, 상태판
data/forecast_30min.csv	30분 구간별·큐별 인입 예측	시뮬레이터, 차트
data/check_10calls.csv	손계산 검증용 10콜	STEP 3 테스트
docs/system_spec.md	요구사항 정의서 — 무엇을 만드나	모든 단계
docs/monitoring_manual.md	관제 매뉴얼 — 정의·임계치·절차	KPI, 경보, AI 요약

어제 이력 calls_yesterday.csv와 단계별 프롬프트 prompts/step_prompts.md도 함께 들어 있다

가상 센터는 큐 3개와 상담사 30명으로 운영된다

분실·도난 신고는 부정 사용 피해와 직결되므로 최우선으로 받는다

큐	우선순위	SL 목표	AHT	받는 팀
분실·도난 신고 (Q1)	1	90% / 15초	180초	분실팀 8명
일반 문의 (Q2)	2	80% / 20초	240초	일반팀 14명 + 지원
대출·한도 상담 (Q3)	3	70% / 30초	360초	대출팀 8명

상담사

- 분실팀·대출팀은 일반 문의도 받는다
- 근무조 09~18시, 10~19시
- 점심은 11:30, 12:30 두 순번

인입

- 10~11시와 14시 무렵 두 번 봉우리
- 봉우리 30분 인입은 190콜 안팎
- 점심 무렵이 가장 한가하다

임계치와 지속 조건은 관제 매뉴얼 3장에서 가져온다

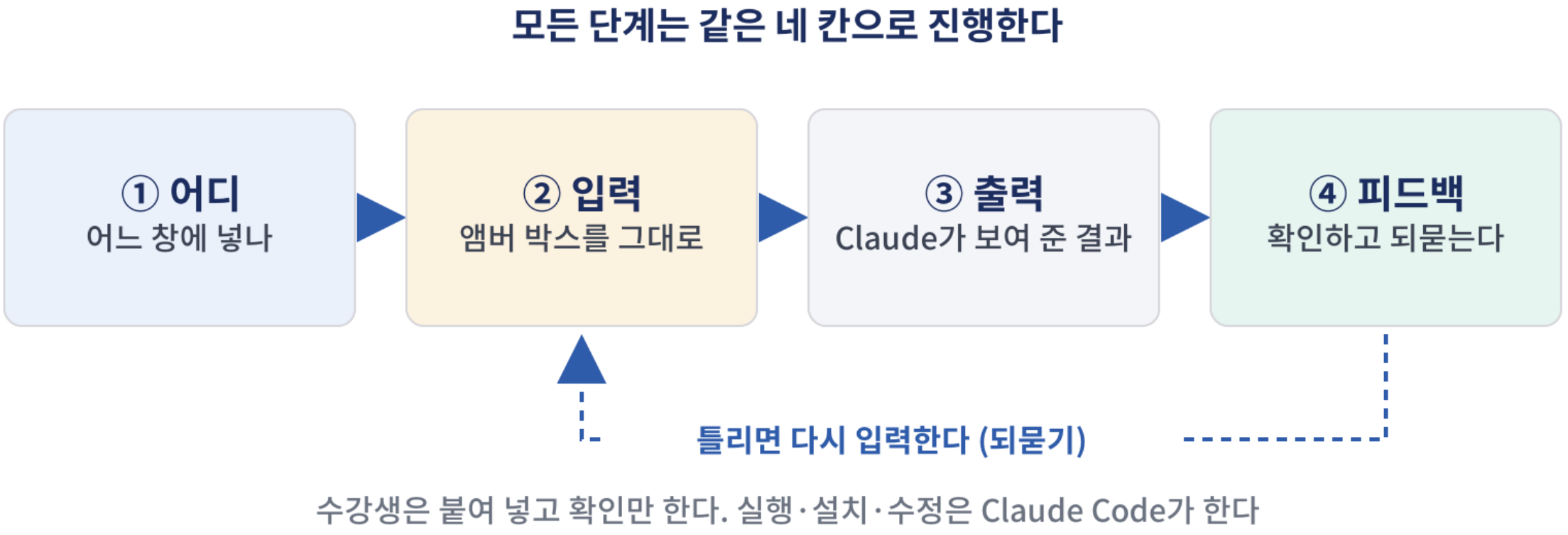
같은 등급 이상이 3분 연속 유지될 때만 경보로 올린다

지표	녹색 (정상)	황색 (주의)	적색 (위험)
서비스레벨	80% 이상	70% 이상 80% 미만	70% 미만
포기율	5% 이하	5% 초과 8% 이하	8% 초과
평균 응답속도	15초 이하	15초 초과 30초 이하	30초 초과
대기 콜	5건 이하	6~12건	13건 이상
최장 대기	60초 이하	60초 초과 120초 이하	120초 초과
점유율	88% 이하	88% 초과 93% 이하	93% 초과

매뉴얼 4장은 황색·인입 급증·인력 감소·적색별 대응 절차를 번호로 정해 둔다(4.1~4.6)

모든 단계는 어디, 입력, 출력, 피드백의 네 칸으로 진행한다

앰버 박스는 그대로 복사해 붙여 넣는 프롬프트, 네이비 박스는 Claude Code가 보여 준 결과다



수강생은 붙여 넣고 확인만 한다. 실행·설치·수정은 Claude Code가 한다

키트 폴더에서 claude를 실행하면 실습이 시작된다

Claude Code 설치와 구독 계정 로그인은 미리 끝내 둔다

[입력] 명령어 · 맥 터미널

```
cd ~/Desktop/callcenter-monitor
```

[입력] 명령어 · 윈도 PowerShell

```
cd $HOME\Desktop\callcenter-monitor
```

[입력] 명령어 · 같은 창에 이어서

```
claude
```

준비 순서

- 바탕화면에 callcenter-monitor 폴더를 만들고 키트 zip 파일을 넣는다.
- 위 명령어로 그 폴더에 들어가 claude를 실행한다. 이후 모든 일은 이 대화창에서 한다.
- 파이썬이 없거나 버전이 낮으면 STEP 0에서 Claude Code가 설치 방법을 안내한다.

열두 단계를 거치며 시스템이 한 층씩 쌓인다

단계마다 실행 결과를 확인하고 git 커밋을 남긴 뒤 다음 단계로 간다



03

단계별 구현

STEP 0~11 프롬프트로 만드는 관제 시스템

단계마다 어디 → 입력 → 출력 → 피드백 순서로 진행한다

STEP 0

키트를 풀고 실행 환경을 갖추는 일도 Claude Code에게 맡긴다

수강생은 압축 파일을 폴더에 넣고 claude 만 실행한다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
키트 압축 파일을 풀고 작업 준비를 해 줘.
1. CallCenter_Monitoring_Kit.zip을 이 폴더에
   푼다.
2. 폴더 구조를 트리로 보여 주고 파일마다
   무엇인지 한 줄로 설명한다.
3. 파이썬 버전을 확인한다(3.10 이상 필요).
4. .venv 가상환경을 만들고 streamlit,
   pandas, altair, pytest를 설치한다.
5. 설치된 패키지 버전을 표로 보여 준다.
```

③ [출력] 결과 예시 · 폴더 트리와 설치 확인

```
data/
  ├ agents.csv        상담사 30명
  ├ queues.csv        큐 3개와 SL 목표
  ├ forecast_30min.csv  구간별 예측
  ├ calls_yesterday.csv 어제 이력
  └ check_10calls.csv   손계산 검증용
docs/  system_spec.md  monitoring_manual.md
prompts/ step_prompts.md
Python 3.12 ✓  streamlit ✓  pandas ✓
altair ✓  pytest ✓
```

④ 피드백

- data 5개, docs 2개 파일이 다 보이는지 확인한다.
- 설치 실패 → "오류 원인을 설명하고 직접 고쳐 줘"

STEP 1

CLAUDE.md에 매뉴얼의 규칙을 먼저 새겨 둔다

CLAUDE.md는 Claude Code가 매번 먼저 읽는 프로젝트 규칙 파일이다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
docs/system_spec.md와 docs/monitoring_
manual.md를 끝까지 읽고 CLAUDE.md를 만들어 줘.
- 프로젝트 목적 한 줄
- 만들 파일과 역할(요구사항 3장)
- 규칙: KPI 정의와 임계치는 매뉴얼 2·3장만
  따른다. 화면과 주석은 한국어로 쓴다.
  시간은 0시 기준 초로 저장한다.
  패키지는 streamlit·pandas·altair만 쓴다.
- 작업 방식: 기능 하나를 만들 때마다
  실행해 확인하고 git에 커밋한다.
다 쓰면 git init 후 첫 커밋을 하고
CLAUDE.md 전문을 보여 줘.
```

③ [출력] Claude가 생성 · CLAUDE.md (예시)

```
# 하늘카드 실시간 관제 시스템
목적: 가상 콜 시뮬레이터와 관제 대시보드
## 규칙
- KPI·임계치: monitoring_manual.md 2·3장
- SL 분모 = 응대 콜 + 포기 콜
- 경보는 같은 등급이 3분 이어질 때만
- 시간: 0시 기준 초 (09:00 = 32400)
## 작업 방식
- 기능마다 실행 확인 → git 커밋
[main (root-commit)] CLAUDE.md 추가
```

④ 피드백

- SL 분모에 포기 콜이 있는지, 임계치가 매뉴얼과 같은지 본다.
- 다르면 → "매뉴얼 3장 표와 한 줄씩 대조해 고쳐 줘"

STEP 2

가상 콜 시뮬레이터가 실제 전화 시스템을 대신한다

예측 인입률로 콜을 만들고 배정·통화·후처리·포기를 매초 처리한다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
요구사항 3장과 5장대로 simulator.py를 만들어 줘.
- data/의 큐·상담사·예측 파일을 읽는다.
- 09:00~18:00 재생. --speed 기본 20(1초=20초)
- 큐별 예측 인입률로 매초 콜을 만들고,
  인내시간을 넘기면 포기로 처리한다.
- 가장 오래 쉰 상담사가 우선순위 높은 큐의
  가장 오래 기다린 콜을 먼저 받는다.
- 통화 뒤 15~45초 후처리를 한다.
- 결과를 live.db의 calls·agents·clock에 쓴다.
- clock의 command 값으로 시나리오를 바꾼다.
다 되면 --speed 2000으로 11:00까지 돌리고
30분 구간별 예측·인입·응대·포기를 표로 보여 줘.
```

③ [출력] 결과 · 30분 구간 점검표

구간	예측	인입	응대	포기
09:00	122	117	111	6
09:30	160	151	139	12
10:00	195	182	176	6
10:30	186	208	195	13

인입이 예측의 90~112% 범위에 있습니다.
포기율은 3~8% 수준입니다.

④ 피드백

- 인입이 예측의 ±15% 안쪽인가? 포기 0건이나 절반 이상은 오류다.
- disk I/O error → "live.db-wal·-shm 파일도 지우게 고쳐 줘"

KPI 엔진의 정답은 10콜 손계산으로 먼저 정해 둔다

check_10calls.csv — 일반 문의(기준 20초) 10콜, 응대 7건과 포기 3건

응대 대기시간: 5, 12, 18, 25, 8, 40, 15초 · 포기 3건

$$\text{서비스레벨} = \frac{5}{7+3} = 50.0\% \qquad \text{포기율} = \frac{3}{10} = 30.0\%$$

$$\text{ASA} = \frac{5+12+18+25+8+40+15}{7} = \frac{123}{7} \approx 17.6\text{초}$$

20초 이내 응대는 5, 12, 18, 8, 15초의 5건이다

- 이 세 숫자를 테스트의 정답으로 고정하고, 코드가 정답을 맞출 때까지 Claude Code가 고치게 한다.

STEP 3

KPI 엔진은 손계산 테스트를 통과해야 믿을 수 있다

10콜 손계산 정답을 테스트로 고정하고 코드가 그 정답을 맞추게 한다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
매뉴얼 2장 정의대로 kpi.py를 만들어 줘.
- 최근 30분 동안 결과가 난 콜로 SL·ASA·
  포기율·AHT를 계산한다.
- 대기 콜·최장 대기·점유율은 지금 값이다.
- SL은 콜마다 자기 큐의 기준 초를 쓴다.
- 시뮬레이터가 1분마다 결과를 kpi_history
  테이블에 남기게 고친다.
그리고 data/check_10calls.csv로 pytest
테스트를 만들어 줘. 정답은 내가 손으로 구한
SL 50.0%, ASA 17.6초, 포기율 30.0%다.
실패하면 코드를 고치고 정답은 바꾸지 마.
```

③ [출력] 결과 · pytest

```
tests/test_kpi.py::test_sl       PASSED
tests/test_kpi.py::test_asa      PASSED
tests/test_kpi.py::test_abandon  PASSED

3 passed in 0.01s

simulator.py: 1분마다 kpi_history 기록 추가
[main 4c1e2a0] KPI 엔진과 손계산 테스트
```

④ 피드백

- 세 테스트가 모두 통과해야 다음 단계로 간다.
- 확인 → "정답 숫자를 바꾸지 않았는지 git diff로 보여 줘"

STEP 4

대시보드는 신호등 타일 여섯 개부터 세운다

임계치는 thresholds.json에 두어 매뉴얼이 바뀌면 파일만 고친다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
app.py를 Streamlit으로 만들어 줘.
이번 단계는 화면 맨 윗부분만 만든다.
- 제목 줄: 가상 시각과 현재 시나리오
- KPI 타일 6개를 한 줄로 놓는다.
- 타일 배경은 매뉴얼 3장 기준으로
  녹색 #2E9E6B, 황색 #E0A100, 적색 #D64545
- 임계치는 thresholds.json에 두고
  코드에 숫자를 직접 쓰지 않는다.
- st.fragment로 2초마다 자동 갱신한다.
시뮬레이터를 백그라운드로 켜고 대시보드를
실행한 뒤 브라우저 주소를 알려 줘.
```

③ [출력] Claude가 생성 · thresholds.json

```
"sl":  {"direction": "high",
         "yellow": 0.80, "red": 0.70},
"abandon_rate": {"direction": "low",
         "yellow": 0.05, "red": 0.08},
"occupancy": {"direction": "low",
         "yellow": 0.88, "red": 0.93},
...
Local URL: http://localhost:8501
```

④ 피드백

- 숫자가 2초마다 바뀌고, 타일 색이 매뉴얼 표와 맞는가?
- 제목이 잘리면 → "화면 위 여백을 늘려 줘"

STEP 5

큐 현황과 상담사 상태판이 원인을 가리킨다

타일이 '무엇이 나쁜가'를 말하면 아래 화면은 '왜 나쁜가'를 보여 준다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
요구사항 4장 3~5번 화면을 app.py에 더해 줘.
- 큐별 현황 표: 대기, 최장 대기, SL,
  SL 목표, 포기율(최근 30분)
- 상담사 상태판: 30명 카드에 이름, 상태,
  그 상태로 머문 시간(분)을 쓴다.
  대기 초록, 통화 파랑, 후처리 연파랑,
  휴식 회색, 장애 이탈 빨강으로 칠한다.
- 30분 구간 인입 차트(altair):
  예측은 회색 점선, 실적은 네이비 실선
화면을 새로 고친 뒤 무엇이 추가됐는지
위치별로 알려 줘.
```

③ [출력] 결과 예시 · 변경 보고

```
추가한 화면 (위에서 아래로)
1. 큐별 현황 표 — 타일 바로 아래 왼쪽
2. 상담사 상태판 — 30장, 한 줄 11장
3. 30분 구간 인입 차트 — 맨 아래 왼쪽

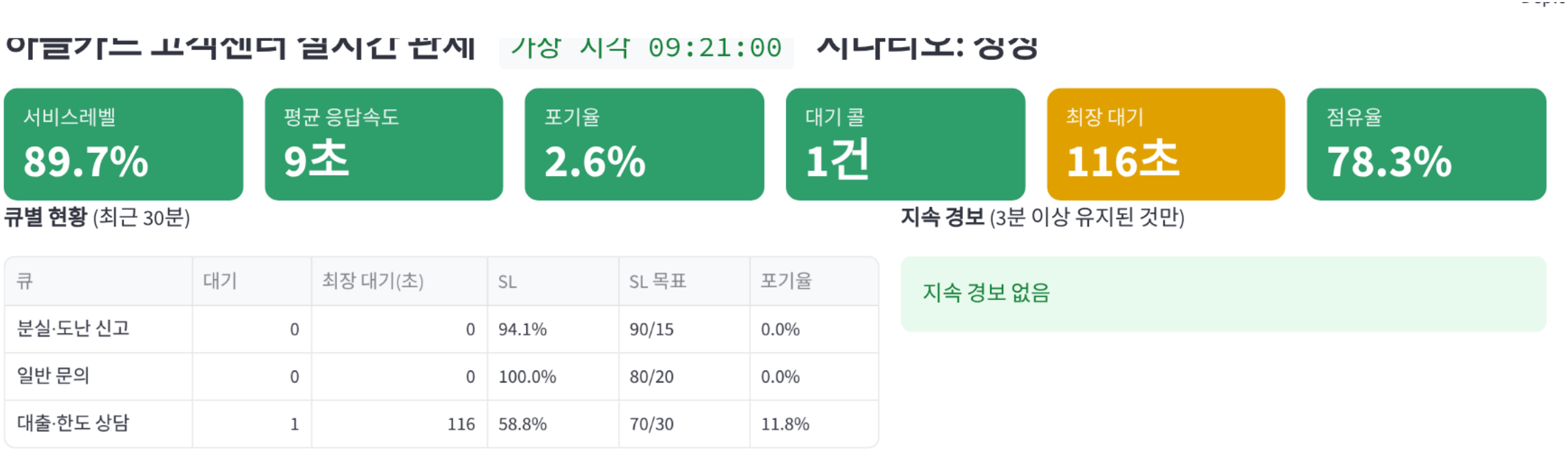
10~19시 근무조 7명은 10:00 전까지
'오프라인'으로 표시됩니다.
[main 9b07d13] 큐 현황·상태판·차트 추가
```

④ 피드백

- 카드가 30장이고, 10시 전 오프라인 인원이 근무표와 맞는가?
- 실적선이 예측선과 크게 어긋나면 STEP 2 표를 다시 본다.

STEP 5가 끝나면 신호등과 큐 현황이 움직이기 시작한다

정상 시나리오 · 가상 시각 09:21 — 제목 줄이 위로 잘린 것은 STEP 4 피드백으로 고친다



큐	대기	최장 대기(초)	SL	SL 목표	포기율
분실·도난 신고	0	0	94.1%	90/15	0.0%
일반 문의	0	0	100.0%	80/20	0.0%
대출·한도 상담	1	116	58.8%	70/30	11.8%

STEP 6

실행 스크립트 하나로 매번 같은 출발점에서 시작한다

시뮬레이터와 대시보드를 따로 켜다 보면 지난 DB가 남아 숫자가 섞인다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
시뮬레이터와 대시보드를 한 번에 켜는
실행 스크립트를 만들어 줘.
- 맥용 run.sh, 윈도용 run.bat
- 이미 켜진 시뮬레이터가 있으면 먼저 끈다.
- live.db는 실행할 때마다 새로 만든다.
- SPEED 값으로 재생 속도를 바꿀 수 있다.
README.md에 실행 방법을 세 줄로 적고
run.sh를 직접 실행해 동작을 확인해 줘.
```

③ [출력] Claude가 생성 · run.sh

```
#!/bin/bash
cd "$(dirname "$0")"
pkill -f "python.*simulator.py"
python3 simulator.py --speed ${SPEED:-20} &
SIM=$!
trap "kill $SIM" EXIT
sleep 2
streamlit run app.py
```

④ 피드백

- 껐다 켜면 가상 시각이 09:00부터 다시 시작하는가?
- 강의 시연은 SPEED=60 ./run.sh 처럼 빠르게 돌린다.

STEP 7

경보는 3분 지속 조건을 걸어야 깜박이지 않는다

매뉴얼 3장: 같은 등급 이상이 3분 연속 유지될 때만 경보로 올린다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
매뉴얼 3장 지속 조건을 alerts.py로 만들어 줘.
- kpi_history 최근 3분이 모두 황색 이상인
  지표만 경보로 올린다.
- 3분 동안 가장 약했던 등급을 경보 등급으로
  쓴다.
- 대시보드 오른쪽에 '지속 경보' 목록을
  등급 색과 함께 보여 준다.
검증 테스트도 만들어 줘. SL이 0.65, 0.68,
0.72인 3분 기록이면 황색 경보 1건, 마지막
2분만 넣으면 경보 0건이 나와야 한다.
```

③ [출력] 결과 · 경보 테스트

```
3분 기록 → [서비스레벨, yellow]
2분 기록 → []  (지속 조건 미달)

test_alert_three_minutes   PASSED
test_alert_two_minutes     PASSED

0.65·0.68은 적색이지만 0.72가 황색이라
3분 내내 유지된 등급은 황색입니다.
```

④ 피드백

- 적색 2분과 황색 1분이 왜 '황색 경보'인지 설명할 수 있는가?
- 깜박이면 → "지속 조건이 kpi_history를 쓰는지 확인해 줘"

STEP 8

시나리오를 주입해 경보가 언제 울리는지 잰다

인입 급증은 10:00~11:00, 상담사 8명 이탈은 14:00~15:00에 넣는다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
사이드바에 시나리오 버튼 3개를 달아 줘.
정상, 인입 급증, 상담사 이탈.
누르면 clock 테이블 command에 이름을 쓴다.
그다음 --speed 200으로 하루를 돌리면서
10:00 인입 급증 → 11:00 정상,
14:00 상담사 이탈 → 15:00 정상을 자동으로
넣고, 지표가 처음 적색이 된 시각과
첫 지속 경보 시각을 표로 정리해 줘.
```

③ [출력] 결과 · 시나리오별 반응 시각

```
인입 급증 (10:00 주입)
  점유율 적색 10:03   포기율 적색 10:04
  대기 콜 적색 10:06  SL 적색     10:08

상담사 이탈 (14:00 주입)
  점유율 경보 14:04   SL 황색     14:32
  SL 경보     14:38   SL 적색     14:47
※ 시뮬레이션 결과, 실행마다 조금 다르다
```

④ 피드백

- 상담사 이탈 때 SL 경보가 점유율보다 34분 늦은 이유를 묻는다.
- → "SL이 늦게 반응하는 이유와 선행 지표를 정리해 줘"

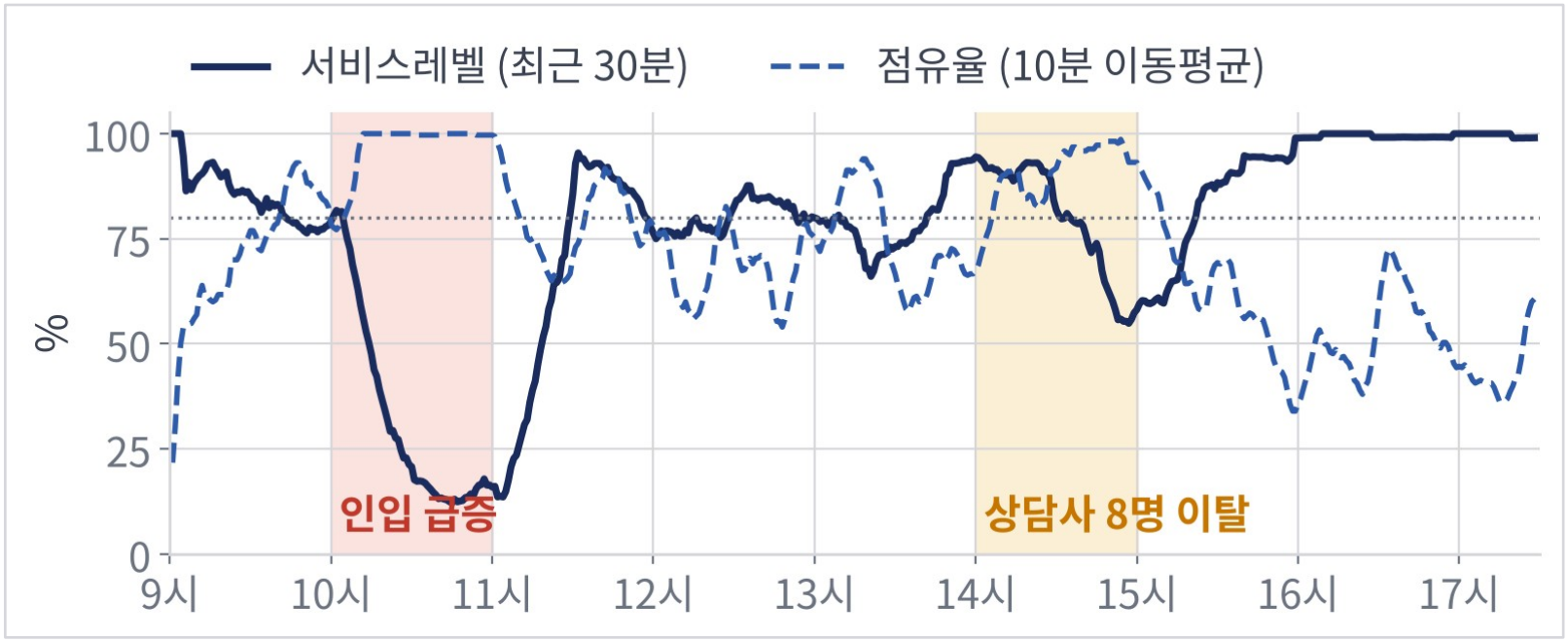
선행 지표

인력 이탈은 점유율이 먼저 알리고 서비스레벨은 늦게 알린다

STEP 8 하루 재생 결과 — 10시 인입 급증, 14시 상담사 8명 이탈 (시뮬레이션)

무엇을 보나

- 급증: 10분 안에 전부 적색
- 이탈: 점유율 경보 14:04
- SL 경보는 14:38
- SL은 30분 평균이라 늦다



인입 급증을 넣으면 신호등이 몇 분 만에 적색으로 바뀐다

인입 1.8배 주입 후 50분 · 가상 시각 11:08 — 분실·도난 큐 SL마저 28%로 떨어졌다

하늘카드 고객센터 실시간 관제 가상 시각 11:08:30 시나리오: 인입 급증

서비스레벨	평균 응답속도	포기율	대기 콜	최장 대기	점유율
37.8%	32초	26.0%	5건	213초	100.0%

큐별 현황 (최근 30분)

큐	대기	최장 대기(초)	SL	SL 목표	포기율
분실·도난 신고	2	16	28.1%	90/15	31.6%
일반 문의	2	23	48.6%	80/20	14.8%
대출·한도 상담	2	206	0.0%	70/30	74.0%

지속 경보 (3분 이상 유지된 것만)

- RED 서비스레벨 36.3%
- RED 포기율 27.7%
- YELLOW 평균 응답속도 31초
- RED 최장 대기 176초
- RED 점유율 100.0%

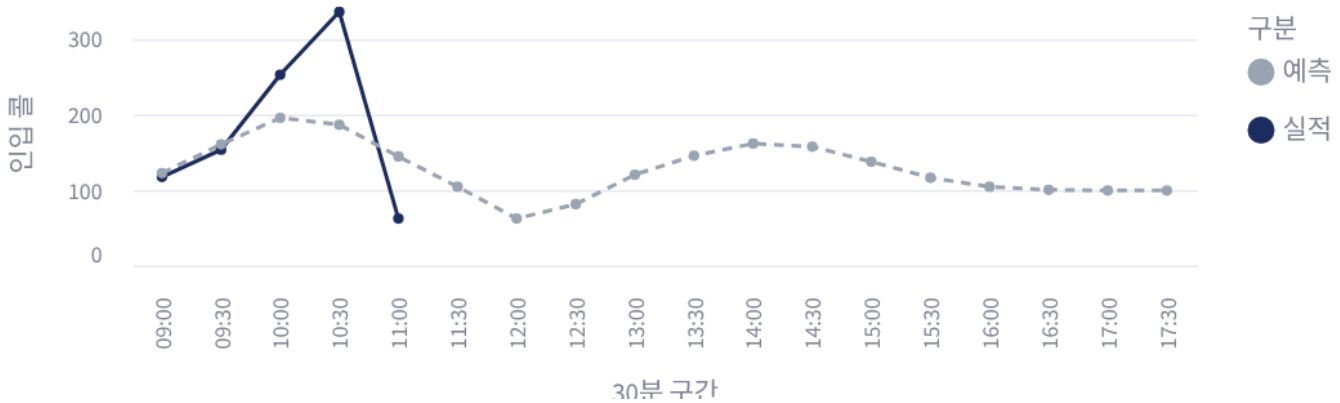
12명이 부족한데 화면은 초록 위쪽 화살표로 보여 준다

같은 순간의 아래쪽 화면 — 이런 표시 오류는 STEP 9의 피드백으로 Claude Code에게 고치게 한다

상담사 상태판

한하늘 통화 1분	서도윤 통화 2분	박서연 후처리 0분	오지호 통화 2분	안민준 통화 2분	이수아 통화 3분	신예린 통화 0분	박현우 통화 1분	박지민 통화 3분	이유나 통화 0분	윤태윤 통화 1분
류다은 통화 0분	이시우 통화 2분	이하린 통화 3분	임준서 통화 2분	정채원 통화 0분	류건우 통화 8분	송소율 통화 2분	최은호 후처리 0분	오나연 통화 1분	송지안 통화 6분	류우진 통화 0분
홍서진 통화 4분	황가온 통화 1분	한로운 통화 1분	류보민 통화 0분	오세아 후처리 0분	윤한결 통화 1분	윤이안 통화 1분	류윤슬 통화 0분			

30분 구간 인입: 예측과 실적



Erlang C 인력 산정 (현재 구간)

예측 기준 필요 인원

23명

현재 인입 속도 기준 필요 인원

42명

↑ 가용 30명 대비 -12명

Erlang C가 부족한 인원을 숫자로 말해 준다

부하(얼랑) = 인입률 × 평균 처리시간. 같은 부하에서 인원별 SL을 계산한다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
erlang.py로 Erlang C 인력 산정을 만들어 줘.
- 입력: 30분 인입 수, AHT(초), 기준 초,
  SL 목표
- 출력: SL 목표를 맞추는 최소 상담사 수
손계산과 맞춰 보게, 30분 160콜·AHT 280초·
기준 20초에서 상담사 25~32명일 때
SL 예측값을 표로 보여 줘.
그다음 대시보드 오른쪽 아래에 예측 기준
필요 인원, 현재 인입 속도 기준 필요 인원,
가용 인원과의 차이를 넣어 줘.
```

③ [출력] 결과 · python erlang.py

```
부하 = 160 × 280 / 1800 = 24.9 얼랑
상담사  25    26    27    28
SL     3.4% 29.8% 49.7% 64.6%
상담사  29    30    31    32
SL    75.5% 83.3% 88.9% 92.7%

SL 80% 목표 필요 인원: 30명
```

④ 피드백

- 부족한데 초록 ↑ 표시 → "부족하면 빨간 ↓로 보이게 고쳐 줘"
- 25명은 SL 3%, 30명은 83%다. 인원과 SL은 비선형이다.

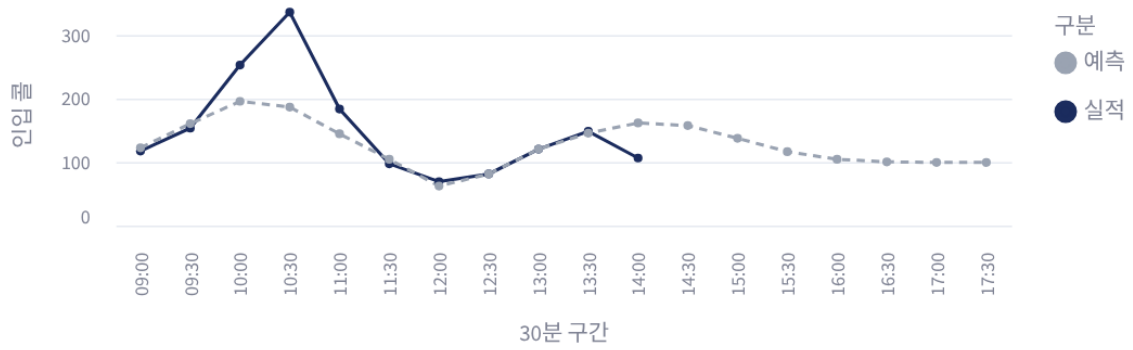
상담사 이탈은 상태판의 빨간 카드와 부족 인원으로 드러난다

이탈 15분 · 가상 시각 14:20 — SL은 아직 녹색, 점유율 95.5% 적색, 부족은 빨간 ↓ 5명으로 고쳐졌다

상담사 상태판

한하늘 통화 1분	서도윤 통화 6분	박서연 후처리 0분	오지호 통화 4분	안민준 통화 3분	이수아 통화 1분	신예린 통화 0분	박현우 통화 1분	박지민 장애 이탈 15분	이유나 장애 이탈 15분	윤태윤 장애 이탈 14분
류다은 장애 이탈 15분	이시우 장애 이탈 14분	이하린 장애 이탈 15분	임준서 장애 이탈 15분	정채원 장애 이탈 15분	류건우 후처리 0분	송소율 통화 0분	최은호 통화 0분	오나연 통화 3분	송지안 후처리 0분	류우진 통화 2분
홍서진 통화 2분	황가온 통화 0분	한로운 통화 0분	류보민 후처리 0분	오세아 대기 0분	윤한결 통화 3분	윤이안 통화 1분	류윤슬 통화 2분			

30분 구간 인입: 예측과 실적



Erlang C 인력 산정 (현재 구간)

예측 기준 필요 인원

28명

현재 인입 속도 기준 필요 인원

27명

가용 인원과의 차이

22명 가용

↓ -5명

STEP 10

AI 요약은 매뉴얼 절차 번호를 근거로 달아야 쓸 수 있다

claude -p 는 Claude Code를 대화창 없이 한 번 실행해 답만 받는 명령이다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
ai_summary.py를 만들어 줘.
- 대시보드의 현재 상황(KPI, 경보, 상태별
  인원, Erlang C 필요 인원)을 JSON으로 묶는다.
- 매뉴얼 4장 전문을 붙여 claude -p로 보낸다.
- 지시: 상황 요약 3줄, 조치 최대 3개(조치마다
  근거 절차 번호), 15분 뒤 볼 지표 1개.
  매뉴얼에 없는 조치는 쓰지 않는다.
- claude 실행이 실패하면 규칙 기반 요약을 쓴다.
사이드바에 'AI 관제 요약 받기' 버튼을 달고
14:40 상담사 이탈 상황으로 실행해 보여 줘.
```

③ [출력] 결과 · claude -p 실제 응답을 줄인 것

```
1) 포기율 10.6% 적색, 점유율 100% 적색,
   SL 77.6% 황색. 장애 이탈 8명으로 가용
   22명, 필요 27명 대비 5명 부족.
2) ① 5분 안에 센터장 보고 (4.4-1, 4.3-4)
   ② 콜백 안내, 비긴급 아웃바운드 중단 (4.4-2)
   ③ IT 상황실 신고, 스킬 확장, 팀장 응대.
      Q1 인력은 빼지 않음 (4.3, 4.5)
3) 15분 뒤 서비스레벨 재확인 (4.6)
```

④ 피드백

- 조치마다 절차 번호가 있는가? 없으면 → "근거 없는 조치는 지워 줘"
- API 키는 필요 없다. 구독으로 로그인한 Claude Code를 부른다.

STEP 11

완료 기준을 Claude Code가 직접 실행해 증명하게 한다

요구사항 7장 완료 기준 5개를 실행 결과로 하나씩 확인한다

① 어디 — Claude Code 대화창 — 키트 폴더에서 claude 를 실행한 터미널

② [입력] 프롬프트 · 그대로 복사해 붙여 넣기

```
요구사항 7장 완료 기준 5개를 하나씩 직접
실행해 확인하고 결과를 표로 보여 줘.
열: 기준, 확인 방법, 결과, 근거
미통과 항목은 고친 뒤 다시 확인해 줘.
끝나면 README.md를 갱신하고 커밋해 줘.
```

③ [출력] 결과 예시 · 완료 기준 점검표

```
1 2초 갱신          통과  10초에 3분 진행
2 손계산 테스트      통과  3 passed
3 급증 SL 적색       통과  10:08 적색
4 이탈 빨간 카드     통과  8장, 부족 5명
5 AI 근거 번호       통과  4.3·4.4·4.6

[main e51f8a2] 완료 기준 점검, README 갱신
```

④ 피드백

- 근거 칸에 실제 숫자·로그가 있는가? '확인함'뿐이면 다시 시킨다.
- 요구사항 밖 기능 → "추가 기능은 되돌리고 제안만 해 줘"

완성

완성된 대시보드는 신호등에서 원인과 조치까지 한 화면에 담는다

오후 정상 운영 · 가상 시각 13:56 (시뮬레이션)

위에서 아래로

- 신호등 타일
- 큐 표와 지속 경보
- 상담사 상태판
- 인입 차트
- 필요 인원
- 왼쪽 버튼 영역



막히면 직접 고치지 말고 상황을 말해 Claude Code에게 맡긴다

아래 네 프롬프트로 대부분의 문제를 풀 수 있다

[입력] 프롬프트 · 오류가 났을 때

방금 오류 메시지의 원인을 쉬운 말로 설명하고, 직접 고친 뒤 다시 실행해 확인해 줘.

[입력] 프롬프트 · 화면이 멈췄을 때

대시보드 숫자가 멈춰 있다. 시뮬레이터가 살아 있는지, live.db의 가상 시각이 바뀌는지 확인해 줘.

[입력] 프롬프트 · 숫자가 의심될 때

SL이 이상하다. 최근 30분 콜 10건을 뽑아 손계산 과정과 함께 보여 줘.

[입력] 프롬프트 · 망가졌을 때

마지막 커밋으로 되돌려 줘. 무엇이 사라지는지 먼저 알려 줘.

관제 시스템의 품질은 정의, 검증, 근거의 세 곳에서 정해진다

이 실습에서 직접 확인한 것

1

정의: 매뉴얼을 CLAUDE.md와 thresholds.json에 옮겼다

숫자를 코드에 박지 않았으므로 매뉴얼이 바뀌면 파일만 고친다.

2

검증: 손계산 테스트와 시나리오 주입으로 확인했다

급증은 10분 안에 적색이 됐고, 이탈은 점유율이 SL보다 34분 먼저 알렸다.

3

근거: 부족 인원과 절차 번호로 조치를 정했다

Erlang C가 몇 명이 모자라는지 말하고, AI 요약은 매뉴얼 번호를 달았다.

다음으로 해 볼 것 — 실제 CTI 로그 연결, 큐별 Erlang C, 경보의 메신저 알림